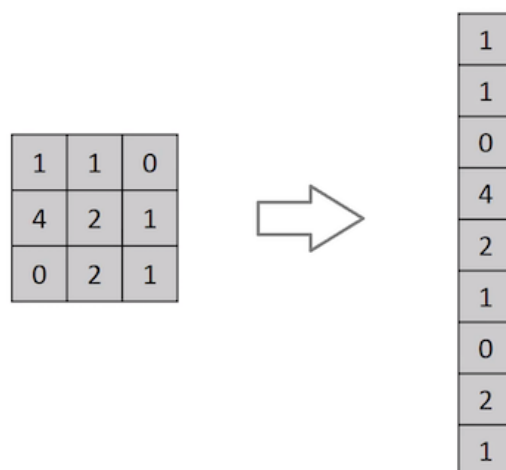


Convolution Neural Networks using PyTorch

Convolutional Neural Nets:

A **Convolutional Neural Network (ConvNet/CNN)** is a Deep Learning algorithm that can take in an input image, assign importance (learnable weights and biases) to various aspects/objects in the image, and be able to differentiate one from the other. The pre-processing required in a ConvNet is much lower as compared to other classification algorithms. While in primitive methods filters are hand-engineered, with enough training, ConvNets have the ability to learn these filters/characteristics.

Why ConvNets over Feed-Forward Neural Nets?



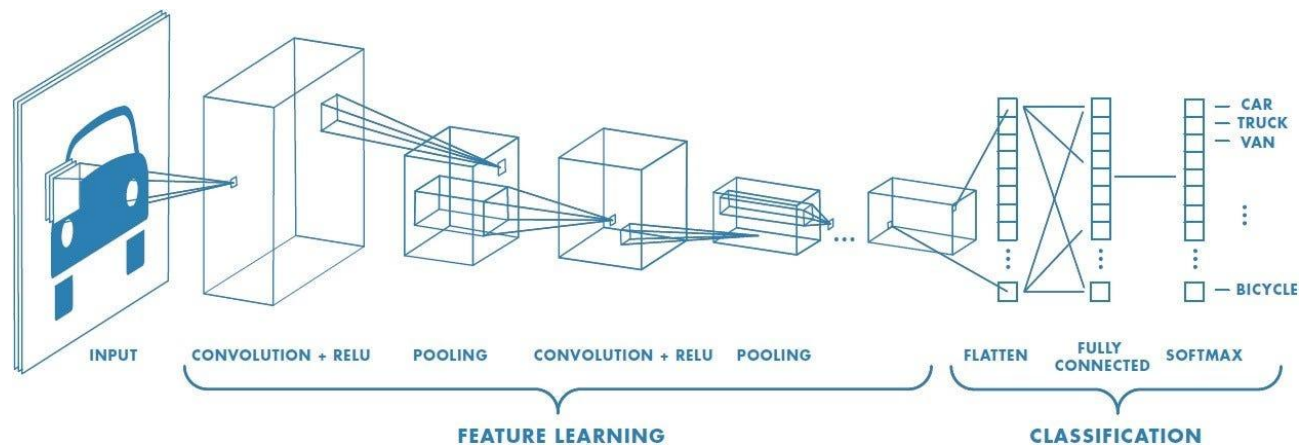
Flattening of a 3x3 image matrix into a 9x1 vector

An image is nothing but a matrix of pixel values, right? So why not just flatten the image (e.g. 3x3 image matrix into a 9x1 vector) and feed it to a Multi-Level Perceptron for classification purposes?

In cases of extremely basic binary images, the method might show an average precision score while performing prediction of classes but would have little to no accuracy when it comes to complex images having pixel dependencies throughout.

A ConvNet can successfully capture the Spatial and Temporal dependencies in an image by applying relevant filters. The architecture performs a better fitting to the image dataset due to

the reduction in the number of parameters involved and the reusability of weights. In other words, the network can be trained to better understand the image's sophistication.



Introduction to Pytorch

PyTorch is a deep learning framework for fast, flexible experimentation. It's completely open source and it gives you a seamless path from prototyping to deploying your neural network models in production. If you're talking about building neural networks, the first framework that you hear of is TensorFlow. That's because TensorFlow is older. PyTorch is relatively new.

PyTorch is a machine learning library for Python built on top of Torch and it was originally developed by artificial intelligence researchers at Facebook. The fundamental data structure that we use to hold the data that we operate on using PyTorch is tensors. Tensors are essentially multidimensional arrays and tensors can run on both the CPU and the GPU. So tensors are very much like NumPy multidimensional arrays. You can perform slicing operations, indexing operations, reductions, linear algebra, matrix, and vector operations, you name it.

Example: Create a CNN classifier on the CIFAR10 dataset using Pytorch

- Install PyTorch and torchvision

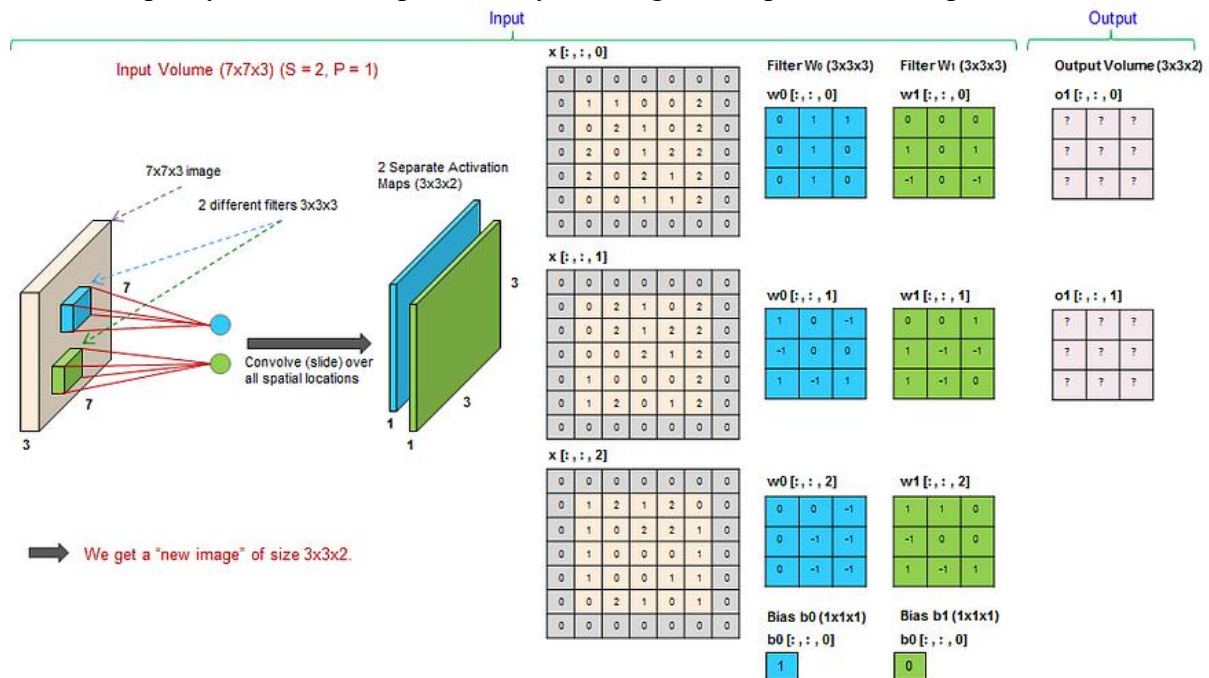
```
!pip install torch
!pip install torchvision
```

Convolutional Layers:

Convolutional layers are the building blocks of CNNs. They consist of several key components:

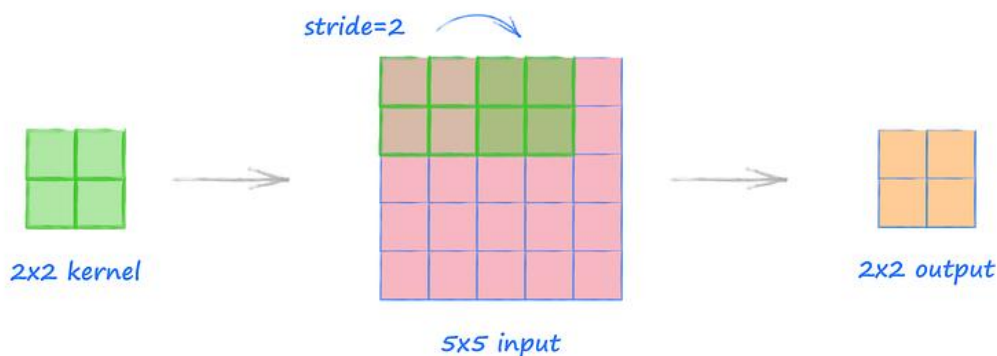
Filters (Kernels):

- Filters are small matrices that slide over the input image and perform element-wise multiplications followed by summation. Each filter is designed to detect a specific feature in the input image.
- For example, a filter might detect horizontal edges, vertical edges, or more complex textures.
- The output of applying a filter to the input image is called a feature map or activation map. If you have multiple filters, you will get multiple feature maps.



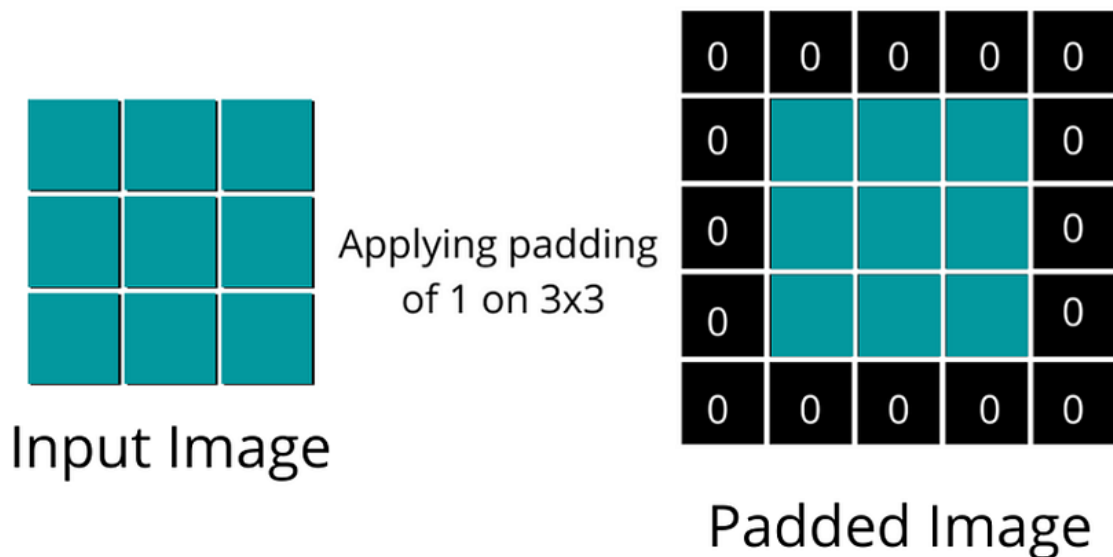
Stride:

- Stride is the step size with which the filter moves across the input image.
- A stride of 1 means the filter moves one pixel at a time, both horizontally and vertically.
- A larger stride reduces the size of the feature map because the filter skips more pixels. For instance, a stride of 2 means the filter moves two pixels at a time, effectively down-sampling the feature map.



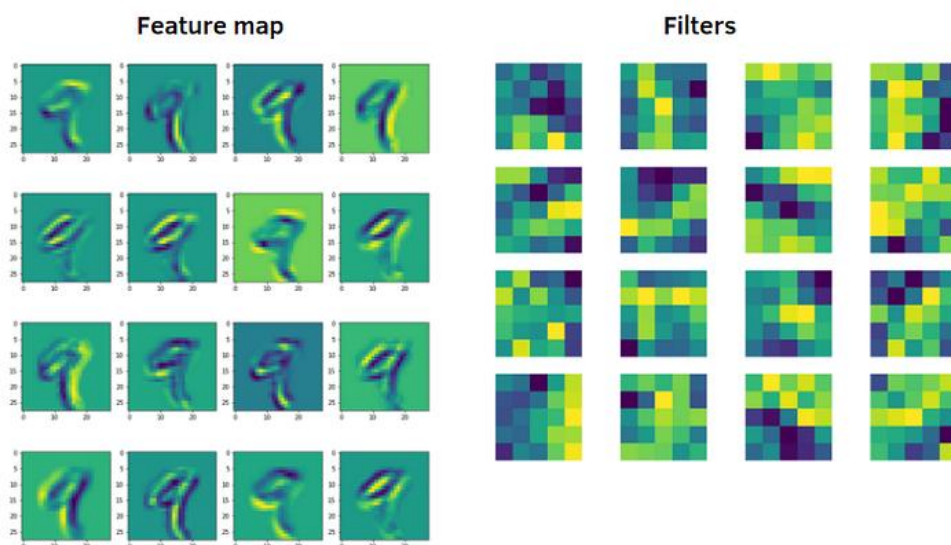
Padding:

- Padding involves adding extra pixels around the input image's border. These extra pixels are typically set to zero (zero-padding).
- Padding ensures that the filter fits properly over the image, especially at the edges. Without padding, the feature map's size reduces after each convolution operation.
- For example, if you have a 5x5 input image and a 3x3 filter with no padding, the resulting feature map will be 3x3. With padding of 1, the feature map remains the same size as the input.



Feature Map:

- A feature map is the output of a convolutional layer after applying filters to the input image.
- Each feature map corresponds to a different filter and captures different features from the input.
- Stacking multiple feature maps together forms a multi-channel output that serves as the input for the next layer.



Pooling Layers

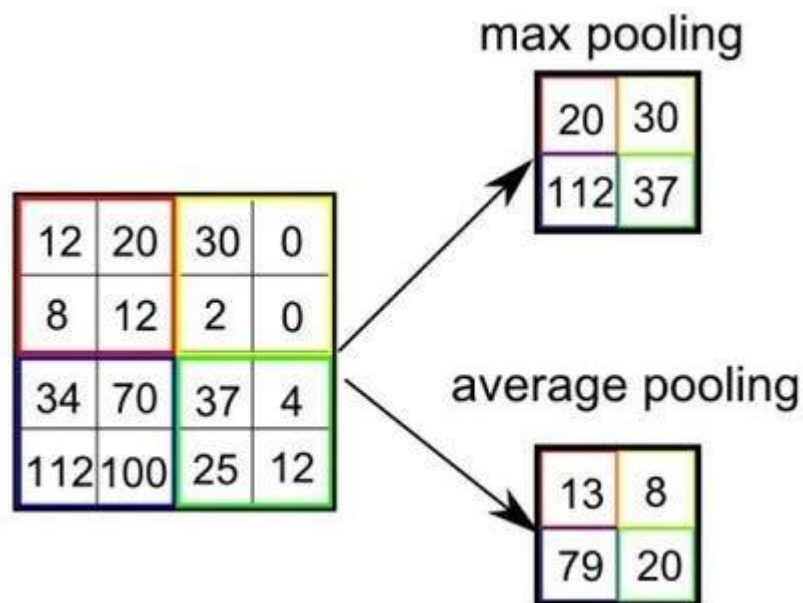
Pooling layers reduce the spatial dimensions of the feature maps, which helps in making the network computationally efficient and reducing overfitting. There are two main types of pooling:

1. Max Pooling:

- Max pooling takes the maximum value from each patch of the feature map.
- For example, in a 2x2 max pooling operation, the maximum value from each 2x2 block of the feature map is taken to create a new, smaller feature map.
- This operation reduces the size of the feature map by half, both horizontally and vertically, but retains the most prominent features.

2. Average Pooling:

- Average pooling takes the average value from each patch of the feature map.
- Similar to max pooling, but instead of the maximum value, it takes the average value from each block.
- This can be useful in different contexts, though max pooling is more common in practice.



In the provided notebook we have a convolutional neural network with 2 Cov layers and a FC Layer with 10 neurons. We are using CIFAR10 dataset in the example. Simply run the complete notebook to train and test the Neural Network that we built.

```

class ConvNet(nn.Module):
    def __init__(self):
        super(ConvNet, self).__init__()

        self.layer1 = nn.Sequential(
            nn.Conv2d(in_size, hid1_size, k_conv_size ),
            nn.BatchNorm2d(hid1_size),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2))

        self.layer2 = nn.Sequential(
            nn.Conv2d(hid1_size, hid2_size, k_conv_size),
            nn.BatchNorm2d(hid2_size),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2))

        self.fc = nn.Linear(hid2_size * k_conv_size * k_conv_size, out_size)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.reshape(out.size(0), -1)
        out = self.fc(out)

        return out

```

Lab Task:

1. Modify the network (ConvNet Class) to have 5 Layers i.e. 4 Conv layers and a FC Layer. You have to figure out what should be the size of parameters for your new layers i.e. **hid1_size**, **hid2_size** variables etc. You have to use flower dataset for this task. The dataset is uploaded in the classroom. You can manually separate some images from each class for testing. Run at least 2-3 Epochs and submit the final notebook named after your RollNo.